Технически университет – София

# Курсова работа

## Тема: Алгоритми за компресия на данни и изображения – LZW

Автор: Алекс Огнянов

Георги Запрянов

**2025г.**

# Съдържание


**1. Общи характеристики** ..... 4
- **1.1. Какво е LZW?** ..... 4
- **1.2. Беззагубна vs. загубна компресия** ..... 4
- **1.3. Какво означава „речников“ алгоритъм** ..... 4

**2. История на LZW** ..... 5
- **2.1. Семейството LZ: 1977–1978** ..... 5
- **2.2. Уелч (1984) и LZW** ..... 5
- **2.3. Масово внедряване и формати** ..... 5
- **2.4. Патентният епизод и ефектът му** ..... 5

***3. Интуиция: как „речникът“ спестява битове*** ..... *6*
- **3.1. Как е възможно декодерът да знае същия речник?** ..... 6

***4. Енкодер (encoder)*** ..... *7*
- **4.1. Термини и означения** ..... 7
- **4.2. Алгоритъм (класически LZW енкодер)** ..... 7
- **4.3. Визуална схема** ..... 8

***5. Декодер (decoder)*** ..... *9*
- **5.1. Основен декодиращ алгоритъм** ..... 9
- **5.2. Какво е „специалният случай“ и защо съществува?** ..... 9
- **5.3. Визуална схема на декодера** ..... 10

***6. Пример стъпка по стъпка*** ..... *11*
- **6.1. Пример вход: ABAABABA** ..... 11
  - **Кодиране (идея)** ..... 11
  - **Декодиране (същите правила)** ..... 11

***7. Вътрешни реализации: структури от данни и битово пакетиране*** ..... *13*
- **7.1. Типични структури за енкодера** ..... 13
  - **Вариант A: Trie (префиксно дърво)** ..... 13
  - **Вариант B: Хеш-таблица за двойки (prefix_code, next_char)** ..... 13
- **7.2. Типични структури за декодера** ..... 13
- **7.3. Битово пакетиране (variable-length codes)** ..... 14
  - **Практически инженерни детайли** ..... 14

***8. Параметри и варианти във формати (GIF, TIFF, PDF, compress)*** ..... *15*
- **8.1. GIF (LZW в GIF89a)** ..... 15
- **8.2. TIFF (LZW в TIFF 6.0 и разширения)** ..... 15
- **8.3. PDF (LZWDecode филтър)** ..... 15
- **8.4. UNIX compress (.Z)** ..... 16

***9. Ефективност, ограничения и типични „капанчета“*** ..... *17*
- **9.1. Кога LZW работи добре?** ..... 17
- **9.2. Кога работи слабо?** ..... 17
- **9.3. Теоретични граници и раздуване** ..... 17

9.4. Чести бъгове при имплементация....17
*10. Приложения и актуалност днес....18*
10.1. Къде се среща LZW....18
10.2. Защо да го учим, ако има „по-нови“ компресии?....18
*11. Свързани алгоритми и какво е „особеното“ при LZW....19*
11.1. LZ77 vs LZ78 vs LZW (в 2 изречения)....19
11.2. Сравнение с Deflate (ZIP/PNG)....19
*12. Сигурност и надеждност при обработка на вход....20*
12.1. Потенциални рискове....20
12.2. Добри практики....20
*13. Заключение....21*
*14. Глосар....21*
15. Източници и препратки....22
15.1. Научни публикации (първични)....22
15.2. Патентни документи (исторически контекст)....22
15.3. Спецификации на формати....22
15.4. Качествени вторични/обяснителни материали....22
Приложение A: Минимален примерен код (псевдо-Python, учебен)....23
Приложение B: Малка „ментална карта“ на процеса....24

# 1. Общи характеристики

## 1.1. Какво е LZW?

**LZW** е универсален беззагубен речников алгоритъм за компресия. Създаден от **Abraham Lempel**, **Jacob Ziv** и **Terry Welch**. Той надгражда **LZ78** алгоритъма и е публикуван за първи път през **1984**.

## 1.2. Беззагубна vs. загубна компресия

Беззагубна **(lossless)**: след декомпресия получаваме абсолютно същите байтове като оригинала. Подходящо за текст, програмен код, документи, някои типове изображения (например с малко цветове), архивиране.

**Загубна (lossy)**: допуска „контролирана“ загуба на информация срещу много по-висок коефициент на компресия (напр. JPEG за снимки, MP3/AAC за звук).

## 1.3. Какво означава „речников“ алгоритъм

Речниковите (dictionary-based) алгоритми търсят повторения в потока и ги заместват с **препратки към речник** от „фрази“ (последователности).

В най-обща картина:

- **Речник**: таблица код → низ (и при енкодера често и обратната: низ → код).
- **Компресия**: входът се преобразува на поредица от фрази, които вече са в речника; енкодерът изпраща **кодовете**.
- **Декомпресия**: декодерът възстановява същите фрази, защото изгражда **същия речник по същите правила**.

LZW е близък до **LZ78**, но с няколко практически инженерни решения, които го правят изключително популярен в реални формати.

# 2. История на LZW

## 2.1. Семейството LZ: 1977–1978

Двата фундаментални „родителя“ на много модерни компресии са:

- **LZ77** (1977): идея за „плъзгащ се прозорец“ и препратки назад към вече видяни фрази.
- **LZ78** (1978): идея за растящ **речник от фрази**; входът се кодира като „индекс + нов символ“.

## 2.2. Уелч (1984) и LZW

През 1984 г. Тери Уелч публикува „A Technique for High-Performance Data Compression“, където предлага вариант, който днес познаваме като **LZW**:

- вместо да предава „индекс + символ“ (както LZ78), алгоритъмът работи с **цели индекси** за вече известни фрази;
- речникът започва от предварително известен набор (например всички байтове 0–255), което улеснява синхронизацията.

Тази публикация е ключова за практическото разпространение на алгоритъма.

## 2.3. Масово внедряване и формати

LZW става широко използван в края на 80-те и 90-те години в:

- **GIF** изображения (формати GIF87a и GIF89a) – емблематичен случай
- **TIFF** – особено за сканирани изображения и графика
- **PDF** – като филтър за компресиране на потоци (streams) в документа

## 2.4. Патентният епизод и ефектът му

В продължение на години LZW е обвързан с патентни ограничения (в зависимост от държавата и периода). Това исторически стимулира появата и популяризацията на алтернативи като **PNG** (за изображения) и други компресори (Deflate/ZIP).

Днес патентните ограничения за класическия LZW в основните юрисдикции са исторически епизод, но ефектът им върху екосистемата (напр. създаване на PNG) е значим.

# 3. Интуиция: как „речникът“ спестява битове

Нека имаме текст:

ABABABABAB...

Вместо да изпращаме:

- A B A B A B …

LZW много бързо „научава“ фрази като:

- AB
- ABA
- BAB
- ABAB
- …

и започва да заменя думи с **индекси**, които с времето могат да станат средностатистически по-къси, отколкото оригинала.

Ключът е в **адаптивността**:

- Речникът се строи спрямо текста в процесът на анализ, което на практика прави така, че алгоритъмът се нагажда спрямо текста
- не е нужна предварителна статистика, както при Хъфман кодиране

## 3.1. Как е възможно декодерът да знае същия речник?

Енкодерът и декодерът:

1. започват от **еднакъв начален речник** (например всички еднобайтови стойности)
2. обработват текста в една и съща последователност
3. добавят нови фрази в таблицата, следвайки еднакви правила

# 4. Енкодер (encoder)

## 4.1. Термини и означения

4.1.1. **Вход:** поток от символи
4.1.2. **Речник:** D, съдържа съответствие фраза -> индекс
4.1.3. **w:** текущо натрупаният низ (фраза)
4.1.4. **k:** следващият входен символ
4.1.5. **wk:** конкатенация w + k

## 4.2. Алгоритъм (класически LZW енкодер)

Идеята е винаги да държим **най-дългата фраза**, която вече е в речника.

**Псевдокод:**

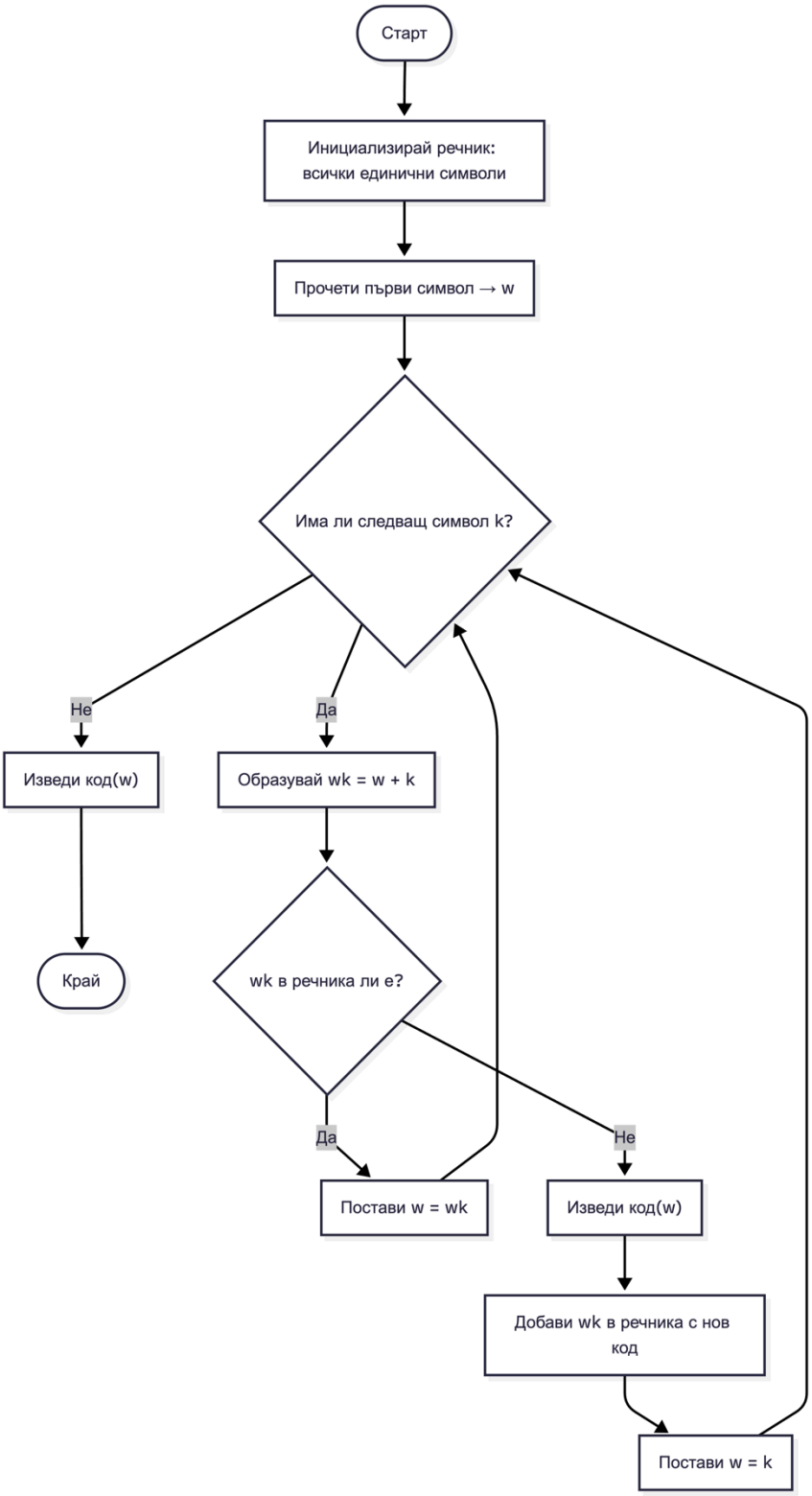
```
инициализирай речник D с всички единични символи (напр. 0...255)
w = празен низ

за всеки следващ символ k от входа:
   ако w е празен:
      w = k
      продължи

   ако wk (w + k) съществува в D:
      w = wk
   иначе:
      изведи код(w)
      добави wk в D с нов код
      w = k

ако w не е празен:
   изведи код(w)
```

## 4.3. Визуална схема



# 5. Декодер (decoder)

Декодерът получава поток от индекси и трябва да възстанови оригиналните символи.

## 5.1. Основен декодиращ алгоритъм

Декодерът поддържа речник D' с мапинг индекс -> фраза(обратното на енкодера).

**Псевдокод:**

```
инициализирай речник D' с всички единични символи (0...255)

прочети първия код c
w = D'[c]
изведи w

за всеки следващ код k:
   ако k съществува в D':
      entry = D'[k]
   иначе ако k е точно следващият свободен код (special case):
      entry = w + първи_символ(w)
   иначе:
      грешка (повреден поток)

   изведи entry
   добави в D' нов запис: (следващ_код) → (w + първи_символ(entry))
   w = entry
```

## 5.2. Какво е „специалният случай“ и защо съществува?

Това е най-често неразбраният детайл.

Понякога енкодерът може да изведе индекс, който декодерът **още не е добавил** в речника си, но който той *трябва* да може да възстанови. Това се случва при модели от типа:

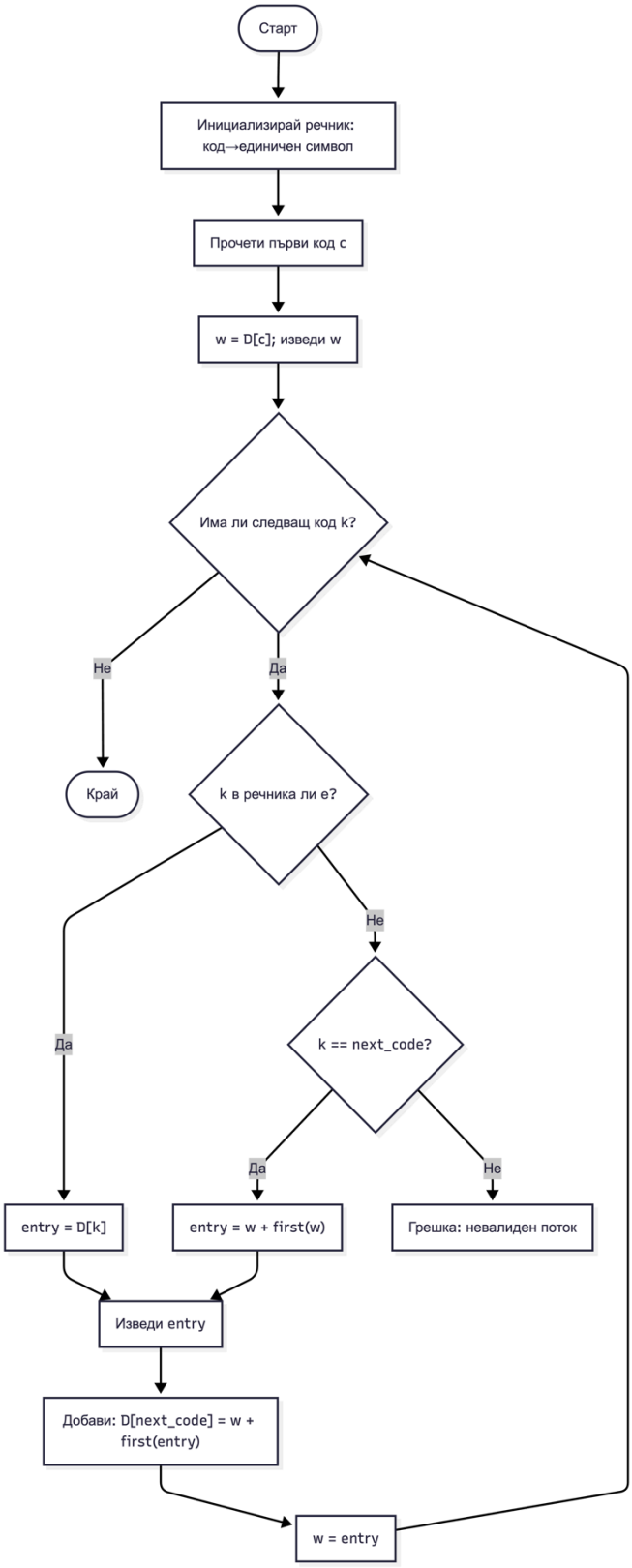
- KwKwK (известен пример)
- където следващата фраза е „предишната фраза + първия ѝ символ“

Интуиция:

- енкодерът добавя нова фраза w + k в момента, в който срещне непозната комбинация
- декодерът добавя по същата логика, но понякога **следващият код** сочи точно към фразата, която „току-що трябва да бъде добавена“

Това не е „магия“, а детерминистичен ефект от начина на синхронно изграждане на речника.

## 5.3. Визуална схема на декодера



# 6. Пример стъпка по стъпка

Ще използваме кратък пример, който показва и „специалния случай“ по-лесно.

## 6.1. Пример вход: ABAABABA

Да приемем азбука от символи A и B (за простота). Реално в байтов поток речникът започва с всички 0…255, но тук ще мислим концептуално.

Начален речник (частично):

- A → 0
- B → 1

Следващите кодове ще са 2, 3, 4, ... за нови фрази.

**Кодиране (идея)**

Демонстрация на основната логика в таблица:

| Стъпка | w (текущо) | k (нов символ) | wk в речника? | Изход | Добавяме в речника |
|---|---|---|---|---|---|
| 1 | A | B | AB – не | A → 0 | AB → 2 |
| 2 | B | A | BA – не | B → 1 | BA → 3 |
| 3 | A | A | AA – не | A → 0 | AA → 4 |
| 4 | A | B | AB – да | – | – |
| 5 | AB | A | ABA – не | AB → 2 | ABA → 5 |
| 6 | A | B | AB – да | – | – |
| 7 | AB | A | ABA – да | – | – |
| 8 | ABA | (край) | – | ABA → 5 | – |

**Компресиран поток (кодове):** 0, 1, 0, 2, 5

**Декодиране (същите правила)**

Начален речник:

- 0 → A
- 1 → B

Четем:

- първи код 0: w = "A", изход: A

Следващ код 1:

- entry = "B" (има го)
- изход: AB

- добавяме w + first(entry) = "A" + "B" = "AB" като код 2
- w = "B"

Следващ код 0:

- entry = "A"
- изход: ABA
- добавяме "B" + "A" = "BA" като код 3
- w = "A"

Следващ код 2:

- entry = "AB" (вече е код 2)
- изход: ABAAB
- добавяме "A" + "A" = "AA" като код 4
- w = "AB"

Следващ код 5:

- но речникът още има записи до код 4; **следващият свободен е 5**
- значи това е **специалният случай**: k == next_code
- entry = w + first(w) = "AB" + "A" = "ABA"
- изход: ABAABABA
- добавяме "AB" + first("ABA") = "AB" + "A" = "ABA" като код 5
- готово

Получихме точно оригинала.

# 7. Вътрешни реализации: структури от данни и битово пакетиране

На теория речникът е „таблица от низове“, но в реален код това е твърде бавно и паметно тежко, ако съхраняваме цели низове като обекти.

## 7.1. Типични структури за енкодера

Енкодерът трябва да отговаря бързо на въпроса:

„Съществува ли фразата wk в речника и какъв е кодът ѝ?“

Две класически решения:

**Вариант A: Trie (префиксно дърво)**

- всяко ребро е символ
- всеки възел може да има код
- търсим най-дългия префикс по символи

**Плюсове:** естествено за „най-дълъг мач“, бързо
**Минуси:** може да е по-паметно, ако азбуката е голяма и имплементацията не е внимателна

**Вариант B: Хеш-таблица за двойки (prefix_code, next_char)**

Много популярно при LZW:

- вместо да пазим целия низ, пазим:
  - prefix_code (код на фразата w)
  - next_char (символ k)
- ключ: (prefix_code, next_char)
- стойност: code(wk)

Това е близко до начина, по който често се описва и в патентната литература и класическите реализации.

## 7.2. Типични структури за декодера

Декодерът често пази речника като два паралелни масива:

- prefix[code] – код на префикса (по-малък код)
- suffix[code] – последният символ

Така фразата за даден code може да се възстанови като се „върви назад“ през prefix докато стигнем базов символ.

За да се изведе фразата в правилен ред, се използва **стек/буфер**:

- тръгваме от code
- докато code не е базов:
    - push suffix[code]
    - code = prefix[code]
- push базовия символ
- pop и печатай

Това е много ефективно: O(дължина на фразата) за извеждане.

## 7.3. Битово пакетиране (variable-length codes)

LZW в реални формати обикновено използва **променлива ширина на кода**:

- в началото кодовете са малки (например 9 бита за 0...511)
- когато речникът расте и се нуждаем от повече кодове, ширината се увеличава (10, 11, 12… бита)
- често има максимум (напр. 12 бита в GIF), за да се ограничи паметта

Това означава, че компресираният поток е **битов** поток, който трябва да се „нареже“ на байтове при запис, а при четене да се сглоби обратно.

**Практически инженерни детайли**

- Нужен е **битов буфер** (например 32-битов регистър) + брояч на валидните битове
- Често формати дефинират конкретен bit-endianness (например LSB-first в GIF)
- Трябва внимателно да се синхронизира моментът, в който се увеличава ширината на кода

# 8. Параметри и варианти във формати (GIF, TIFF, PDF, compress)

Важно: „LZW“ като идея е една и съща, но форматите добавят правила:

- специални кодове (clear/end)
- максимална ширина
- кога се увеличава ширината
- кога и как се рестартира речникът
- битов ред

## 8.1. GIF (LZW в GIF89a)

GIF използва LZW с няколко характерни особености:

- речникът се инициализира с всички „пикселни стойности“ според размер на палитрата (минимален размер на кода);
- присъстват два специални кода:
  - **Clear code**: сигнал за нулиране на речника;
  - **End of Information**: край на данните;
- максимален размер на кода обикновено **12 бита** (т.е. до 4096 записа);
- кодовете се пакетират в байтове по определен ред (описан в спецификацията).

**Практически ефект:** GIF може да се справя добре с изображения с големи области от еднакви/повтарящи се структури (иконки, графики, текстови банери, анимации с малко промени).

## 8.2. TIFF (LZW в TIFF 6.0 и разширения)

TIFF исторически използва LZW като една от компресиите за растерни данни:

- често в комбинация с „predictor“ (например хоризонтално диференциране), което прави входа по-компресируем;
- речникът обикновено се ограничава (често 12 бита);
- данните може да са организирани на **стрийпове** (strips) или **плочки** (tiles), всеки със собствен компресиран поток.

## 8.3. PDF (LZWDecode филтър)

PDF поддържа компресия на поток (stream) чрез филтри, сред които исторически е:

- **LZWDecode** (LZW),

- **FlateDecode** (Deflate),
- други.

PDF добавя идеята за **параметри на декодиране** (DecodeParms), например:

- „предиктори“ (TIFF/PNG предиктори) за по-добро компресиране на изображение-подобни данни;
- опции за момента на увеличаване на кодовата ширина (в някои реализации се среща параметър като EarlyChange).

На практика в модерни PDF-и често се среща FlateDecode, но LZWDecode съществува поради обратна съвместимост.

### 8.4. UNIX compress (.Z)

Исторически UNIX инструментът compress използва LZW-подобна схема с:

- променлива ширина на кода (например до 16 бита в някои реализации),
- възможност за „block mode“/clear код при намаляване на ефективността.

Това е пример как една и съща идея се адаптира към нуждите на потокова компресия на файлове.

# 9. Ефективност, ограничения и типични „капанчета“

## 9.1. Кога LZW работи добре?

LZW се представя силно при данни с:

- **повтарящи се фрази** (текст, логове, структурирани файлове)
- **ограничена азбука** и повторяемост (прости изображения, палитрови графики)
- локални повторения (но по-скоро като последователности, не като „прозоречни“ повторения както LZ77)

## 9.2. Кога работи слабо?

- При данни близо до случайни (висока ентропия): криптирани/компресирани вече файлове, истински шум
- При твърде малки входове: overhead-ът от речник/индекси може да заема повече място, отколкото спестява

## 9.3. Теоретични граници и раздуване

Беззагубните компресори принципно не могат да компресират всички входове; за някои входове е възможно **раздуване** (compressed > original)

Реалните формати се защитават чрез:

- ограничение на речника (максимална кодова ширина)
- clear/reset при пренасищане
- превключване на режим или fallback

## 9.4. Чести бъгове при имплементация

- **Грешен „битов ред“** при ендкориане/декодиране
- **Неправилен момент за увеличаване на ширината** (off-by-one)
- **Пропуснат специален случай** при декодиране (когато кодът е равен на next_code)
- **Неправилно нулиране на речника** при clear code (особено в GIF/TIFF).
- **Buffer overload** при възстановяване на фраза (ако няма лимити и валидиране)

# 10. Приложения и актуалност днес

## 10.1. Къде се среща LZW

- **GIF**: класически анимации и графики в уеб и UI активи
- **TIFF**: сканирания, архивни изображения, печатни работни потоци
- **PDF**: стари документи, някои генератори, специфични потоци
- **Наследени архиви (.Z)**: исторически системи и архиви

## 10.2. Защо да го учим, ако има „по-нови“ компресии?

- LZW е отличен пример за:
  - речниково мислене
  - потокови алгоритми
  - синхронно изграждане на структури без обмен на метаданни
  - битови потоци и форматно-специфична интеграция
- Среща се в реалния свят при:
  - обработка на архиви
  - дигитална консервация
  - форензика/анализ на файлови формати
  - вградени системи и кодови бази с наследство

---

# 11. Свързани алгоритми и какво е „особеното“ при LZW

## 11.1. LZ77 vs LZ78 vs LZW (в 2 изречения)

- **LZ77**: референции назад към „прозорец“ в вече видяното (offset, length)
- **LZ78**: растящ речник от фрази, токени от тип „индекс + символ“
- **LZW**: оптимизация/вариант на LZ78, където се изпращат **кодовете на фразите** и речникът започва от известна база

## 11.2. Сравнение с Deflate (ZIP/PNG)

Deflate комбинира:

- LZ77-подобни референции +
- Хъфман кодиране (ентропийно кодиране)

Това обикновено дава по-добра компресия от чист LZW, но е по-сложна схема.

# 12. Сигурност и надеждност при обработка на вход

LZW сам по себе си не е „опасен“, но декодирането само по себе си е обработка на външен вход, което означава:

## 12.1. Потенциални рискове

- **Decompression bombs**: силно компресиран поток, който заема много ресурси при декодиране (CPU и/или RAM)
- **Повредени/злонамерени потоци**: кодове извън диапазон, липсващи end-кодове, грешни bitstream граници
- **Неправилни лимити**: ако не ограничаваме размера на изхода, рискуваме DoS

## 12.2. Добри практики

- лимит на максимален изход (или потоково извеждане към ограничен sink);
- валидиране на кодове:
  - k < next_code или специалният случай k == next_code, иначе грешка
- контрол върху максималната ширина/размер на речника
- стабилен битов парсер, устойчив на непълни/къси потоци (особено при PDF, където срещаме „реални“ несъвършенства)

## 13. Заключение

LZW е един от най-важните и разбираеми беззагубни алгоритми за компресия:

- разчита на проста, но мощна идея: **речник от фрази + кодове**
- работи потоково и адаптивно
- исторически е оформил цели екосистеми (особено около GIF/TIFF/PDF)
- предоставя отлична основа за разбиране на по-сложни компресии

## 14. Глосар

- **Азбука (alphabet)**: множеството от възможни символи (в практиката често байтове 0…255).
- **Беззагубна компресия**: компресия с точно възстановяване на оригинала.
- **Речник**: таблица, която свързва кодове и низове.
- **Фраза**: последователност от символи, съхранена в речника като единица.
- **Кодова ширина**: броят битове, с които се записва един индекс (например 9, 10, 12).
- **Clear code**: специален код за нулиране на речника (в някои формати).
- **EOI / EOD**: код/маркер за край на информацията/данните (форматно специфично).

# 15. Източници и препратки

## 15.1. Научни публикации (първични)

15.1.1. **Terry A. Welch** (1984). *A Technique for High-Performance Data Compression.* IEEE Computer, 17(6), 8–19. DOI: 10.1109/MC.1984.1659158

- PDF (университетско огледало): A Technique for High-Performance Data Compression
- DOI страница: A Technique for High-Performance Data Compression | IEEE Journals & Magazine

15.1.2. **Jacob Ziv, Abraham Lempel** (1978). *Compression of Individual Sequences via Variable-Rate Coding.* IEEE Transactions on Information Theory, IT-24(5), 530–536.

- PDF (университетско огледало): Compression of Individual Sequences via Variable-Rate Coding

## 15.2. Патентни документи (исторически контекст)

15.2.1. **US Patent 4,558,302**. *High speed data compression and decompression apparatus and method.*

- PDF (изображение на патента): https://patentimages.storage.googleapis.com/2c/fc/25/2b29895a953f8f/US4558302.pdf
- Google Patents: US4558302A - High speed data compression and decompression apparatus and method - Google Patents

## 15.3. Спецификации на формати

15.3.1. **GIF89a Specification** (текстово огледало):

- the GIF89a Specification

15.3.2. **TIFF 6.0 Specification** (PDF огледало):

- Revision 6.0

15.3.3. **ISO 32000-1:2008 (PDF 1.7)** – оторизирано копие (Adobe):

- Portable document format — Part 1: PDF 1.7

## 15.4. Качествени вторични/обяснителни материали

15.4.1. Library of Congress, Format Description: *LZW Compression Encoding*

- LZW Compression Encoding

15.4.2. Library of Congress, Format Description: *TIFF Bitmap with LZW Compression*

- TIFF Bitmap with LZW Compression

## Приложение А: Минимален примерен код (псевдо-Python, учебен)

Това е опростен пример за **идеята** на LZW върху низове. Реални формати (GIF/TIFF/PDF) добавят clear/end кодове, битово пакетиране и ограничения.

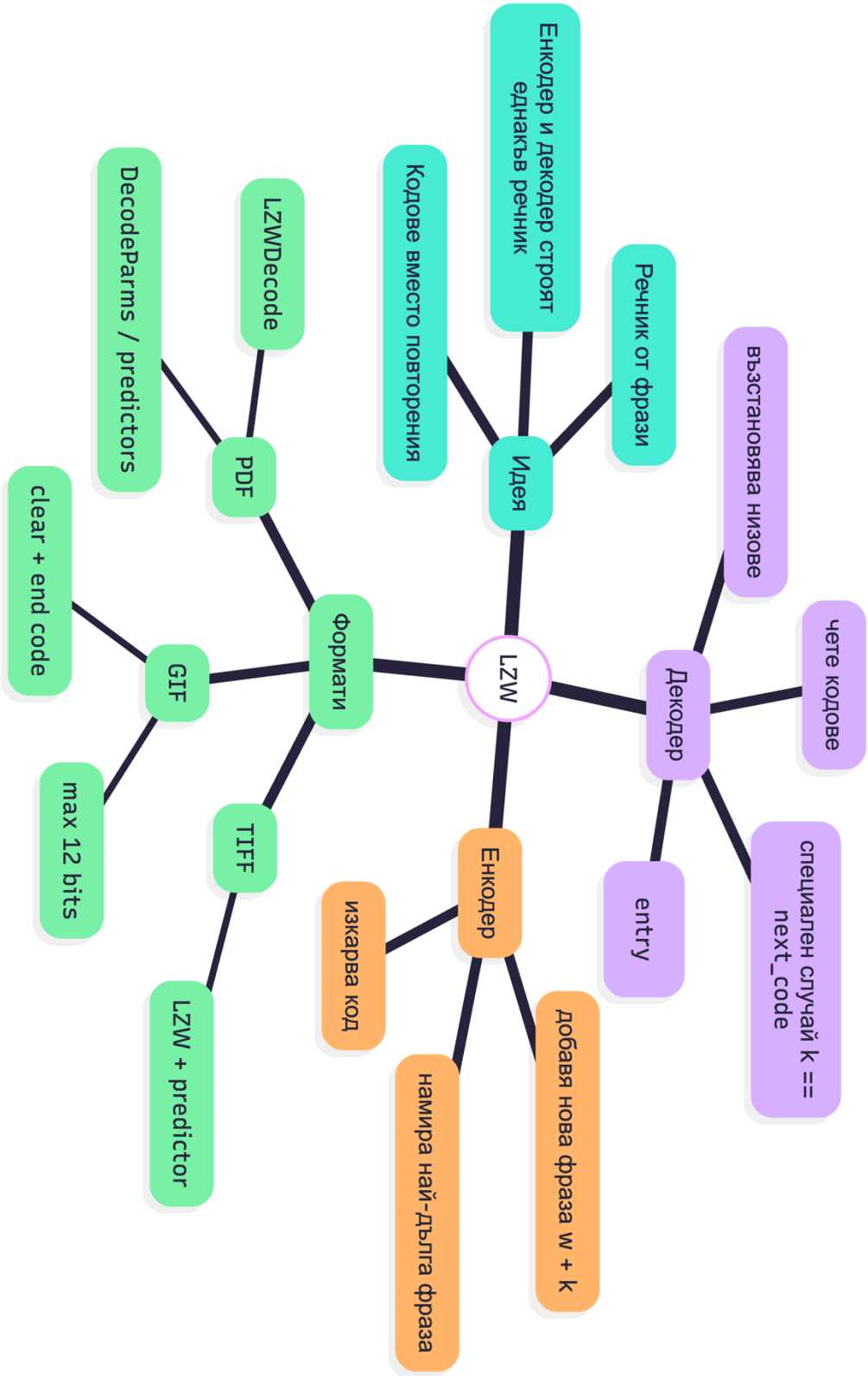
```
def lzw_encode(data: str):
    # Инициализираме речника с единични символи (за пример)
    dictionary = {chr(i): i for i in range(256)}
    next_code = 256

    w = ""
    output = []
    for k in data:
        wk = w + k
        if wk in dictionary:
            w = wk
        else:
            output.append(dictionary[w])
            dictionary[wk] = next_code
            next_code += 1
            w = k
    if w:
        output.append(dictionary[w])
    return output

def lzw_decode(codes):
    dictionary = {i: chr(i) for i in range(256)}
    next_code = 256

    w = dictionary[codes[0]]
    out = [w]
    for k in codes[1:]:
        if k in dictionary:
            entry = dictionary[k]
        elif k == next_code:
            entry = w + w[0]  # специалният случай
        else:
            raise ValueError("Невалиден код")

        out.append(entry)
        dictionary[next_code] = w + entry[0]
        next_code += 1
        w = entry

    return "".join(out)
```

## Приложение В: Малка „ментална карта“ на процеса